

- Per la definizione di *SaveEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.6**.

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.7.2.1) FlaskInsertJustificationController

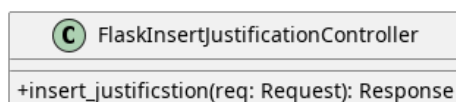


Figura 155: FlaskInsertJustificationController

Descrizione

FlaskInsertJustificationController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di inserimento di una giustificazione per un requisito e le inoltra al livello applicativo.

Attributi

- `insert_justification_use_case`: `InsertJustificationUseCase` — inbound port usata per inserire la giustificazione.

Metodi

- `+ insert_justification(req: Request): Response` — riceve la richiesta HTTP di inserimento della giustificazione, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.7.2.2) InsertJustificationUseCase

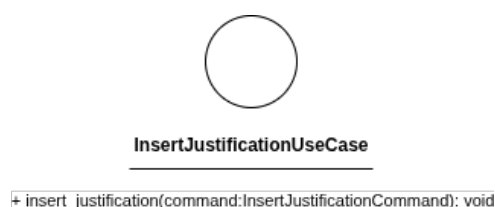


Figura 156: InsertJustificationUseCase

Descrizione

InsertJustificationUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'inserimento di una giustificazione testuale associata a un requisito di conformità. Viene implementata da *InsertJustificationService* e utilizzata da *FlaskInsertJustificationController*.

Attributi

InsertJustificationUseCase non definisce attributi.

Metodi

- + insert_justification(command: InsertJustificationCommand): void — firma del metodo delegato all'esecuzione della logica di inserimento della giustificazione a partire dai dati contenuti nel comando.

5.7.2.3) InsertJustificationCommand

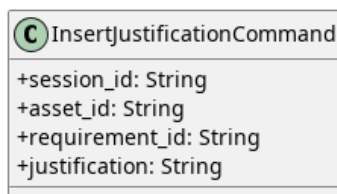


Figura 157: InsertJustificationCommand

Descrizione

InsertJustificationCommand è l'oggetto che veicola i dati necessari all'inserimento di una giustificazione dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione attiva.
- + asset_id: String — identificativo dell'Asset oggetto di valutazione.
- + requirement_id: String — identificativo del requisito a cui si associa la giustificazione.
- + node_id: String — identificativo del nodo decisionale correlato.
- + justification: String — testo della giustificazione da inserire.

Metodi

InsertJustificationCommand non definisce metodi.

5.7.2.4) InsertJustificationService



Figura 158: InsertJustificationService

Descrizione

InsertJustificationService è il service applicativo appartenente all'Application Core responsabile della logica di inserimento di una giustificazione per un requisito di conformità. Implementa l'interfaccia *InsertJustificationUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, associa la giustificazione al requisito specificato e persiste la sessione aggiornata tramite *SaveEvaluationSessionPort*.

Attributi

- `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per recuperare la sessione di valutazione.
- `save_evaluation_session_port`: `SaveEvaluationSessionPort` — outbound port usata per salvare le modifiche applicate alla sessione.

Metodi

- `+ insert_justification(command: InsertJustificationCommand): void` — concretizza il contratto definito da *InsertJustificationUseCase*. Recupera la sessione attiva, individua il requisito corrispondente ai parametri incapsulati nel comando, registra la giustificazione fornita e persiste la sessione aggiornata.

5.8) Architettura Frontend: Widget Semplici

L'approccio server-side rendering adottato nel progetto è integrato da componenti e widget reattivi realizzati con Vue 3 (Composition API). Per ogni pagina, Flask rende-
rizza il template Jinja con i dati statici, mentre le parti interattive vengono delegate a «isole» Vue indipendenti, montate su specifici punti del DOM.

Il sistema frontend gestisce solo l'aspetto grafico e la reattività, altre operazioni vengono realizzate tramite chiamate API.

Per mantenere il codice aderente ai principi SOLID — in particolare Single Responsibility e Dependency Inversion — l'architettura frontend si sviluppa su tre livelli:

3. Componenti Dumb

Componenti UI generici e riutilizzabili, privi di qualsiasi conoscenza del dominio applicativo. Ricevono dati e callback esclusivamente tramite props ed emettono eventi verso il padre. Non accedono a store, non eseguono chiamate di rete, non conoscono la struttura degli URL.

2. Widget Smart

Isole applicative che orchestrano uno o più componenti dumb per realizzare una funzionalità di dominio specifica. Contengono la logica di business (chiamate HTTP, validazione, gestione del redirect post-azione) e compongono i componenti dumb passando loro i dati necessari. Ogni widget è autocontenuto nella propria cartella.

1. Entry Point (Mount Point)

Livello di integrazione tra il mondo Flask/Jinja e Vue. Ogni entry point legge i dati iniettati dal server tramite attributi `data-*` del DOM, li converte nel formato atteso dal widget e monta l'applicazione Vue sul nodo HTML corrispondente. Non contiene logica di business né di presentazione.

Logica Condivisa (Shared) Trasversalmente ai tre livelli, la cartella `shared` contiene logica riutilizzabile che non è legata a un singolo widget né costituisce un componente visuale:

- **Composable** — Funzioni che incapsulano stato reattivo e logica riutilizzabile tra widget diversi. Seguono la convenzione Vue use*. Esempio: useFormModel, che gestisce lo stato dei campi, la validazione client-side e l'integrazione con errori server-side.
- **Definizioni di dominio** — Oggetti che descrivono la struttura dei dati condivisi tra widget. Esempio: deviceFormFields e assetFormFields, che definiscono campi e regole di validazione dei rispettivi form, condivisi tra i widget di creazione e modifica.
- **Utilità** — Funzioni pure senza dipendenze da Vue, utilizzabili sia dai widget che dagli entry point. Esempio: navigationGuard, che gestisce il blocco di navigazione per le pagine con dati non salvati.

Questa separazione garantisce che i componenti dumb siano riutilizzabili tra widget diversi, che i widget siano testabili in isolamento, e che la dipendenza dagli URL e dai dati del server sia confinata esclusivamente nel livello di integrazione.

5.8.1) Shared

5.8.1.1) Validation Rule

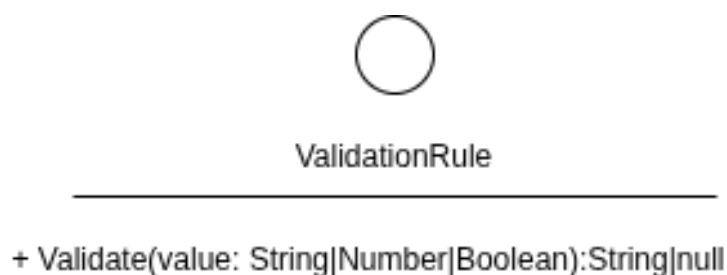


Figura 159: Shared - ValidationRule

Descrizione:

Viene utilizzato per la validazione dei dati inseriti dall'utente. Rappresenta un contratto, un oggetto deve esporre un metodo validate che accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

Attributi:

Le interfacce non hanno attributi

Metodi:

- + validate(value:String|Number|Boolean):String|null: Accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

5.8.1.2) Field Definition

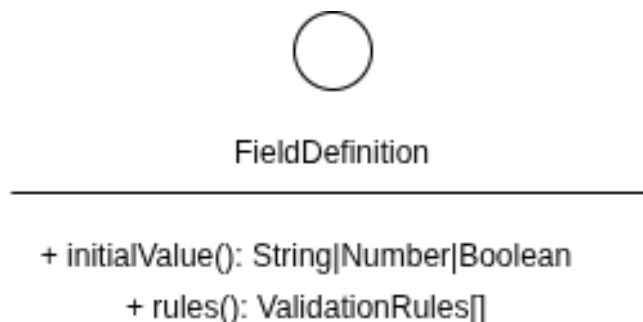


Figura 160: Shared - FieldDefinition

Descrizione: Rappresenta un contratto strutturale che definisce i campi che una oggetto deve esporre per poter essere usato nel model di un form.

Attributi: L'interfaccia non ha attributi.

Metodi:

- + initialValue():String|Number|Boolean:Il valore primitivo di default con cui il campo deve essere inizializzato nel DOM e a cui deve essere ripristinato in fase di reset.
- + rules(): ValidationRule[] :Un array di elementi che implementano l'interfaccia ValidationRule, contenente l'elenco delle funzioni di validazione associate al campo.

5.8.1.3) Use Form Model

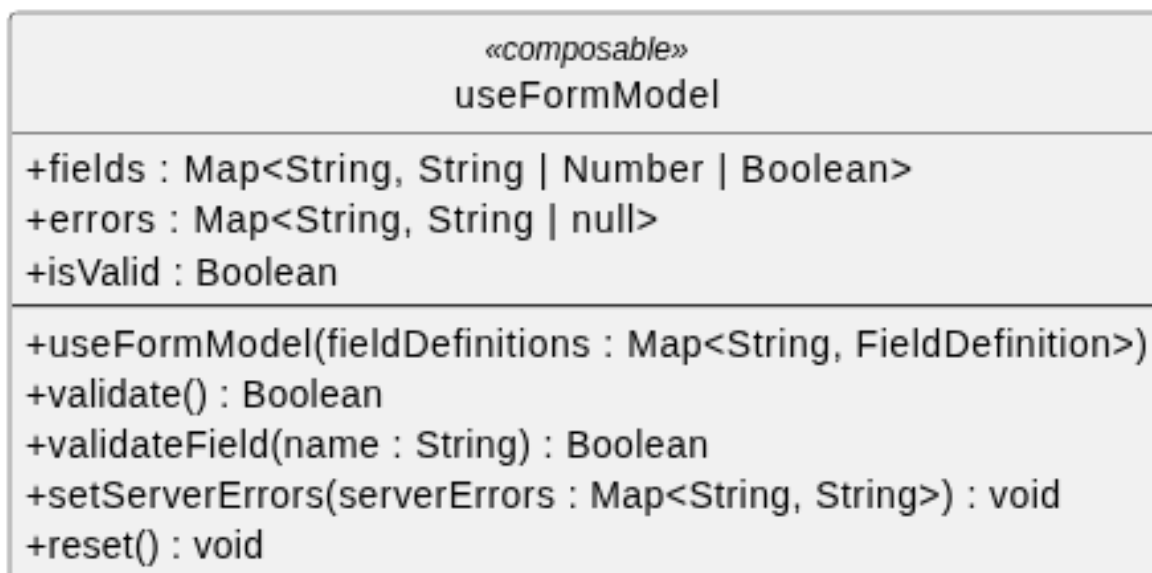


Figura 161: Shared - UseFormModel

Descrizione:

Inizializza e gestisce lo stato reattivo di un form di dominio a partire dalle sue definizioni. Si occupa della validazione client-side e dell'integrazione degli errori inviati dal server, isolando completamente la logica di business dal template visivo.

Attributi:

- - `fieldDefinitions` : `Map<String, FieldDefinition>`: Conserva internamente le field definition ricevute alla costruzione e le usa per usarlo nelle funzioni.
- + `fields`: `Map<String, String | Number | Boolean>` : Mappa reattiva che memorizza i valori correnti inseriti dall'utente per ciascun campo del form.
- + `errors`: `Map<String, String | null>`: Mappa reattiva che associa a ogni campo il relativo messaggio di errore (impostato a null se il campo è valido).
- + `isValid`: `Boolean` : Proprietà calcolata (computed). Restituisce true solo se tutti gli elementi all'interno di `errors` sono pari a null.

Metodi:

- + `useFormModel(fieldDefinitions: Map<String, FieldDefinition>)` : Costruttore/Funzione di inizializzazione. Riceve la configurazione dei campi e genera le strutture reattive per `fields` ed `errors` impostando i valori iniziali.
- + `validate()` : `Boolean` : Esegue la validazione su tutti i campi del form. Restituisce true se l'intero modulo è valido, altrimenti aggiorna la mappa degli errori e restituisce false.
- + `validateField(name: String)` : `Boolean` : Invocato per convalidare un singolo campo (es. all'evento di **blur** o di **input**). Applica le regole in ordine sequenziale e si interrompe al primo fallimento.
- + `setServerErrors(serverErrors: Map<String, String>)` : `void` : Riceve una mappa di errori generati dalle API di Flask (backend) e li inietta direttamente nello stato `errors` per mostrarli all'utente.
- + `reset()` : `void` : Svuota tutti i messaggi di errore e ripristina i valori di `fields` allo stato iniziale definito in `FieldDefinition`.

5.8.1.4) Definizioni di Dominio (Constants)

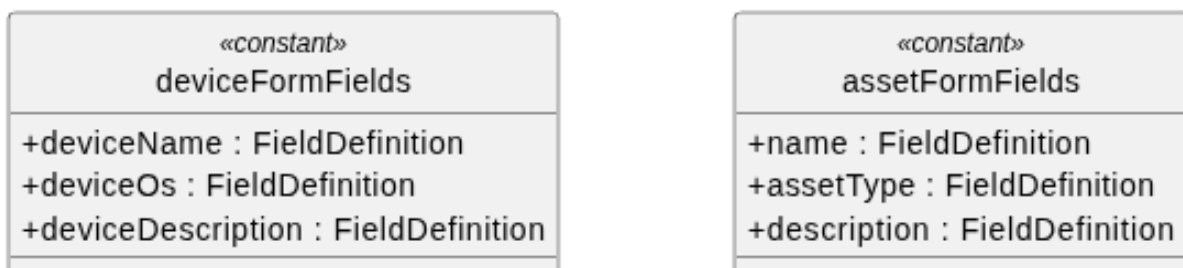


Figura 162: Shared - Form Fields Configurations

Descrizione:

Oggetti JavaScript esportati come costanti, che fungono da singolo punto di configurazione per le regole di validazione degli input utente nei form.

Rappresentano una `map<String, FieldDefinition>`, questi oggetti isolano le regole di validazione) e i valori iniziali dal resto del codice. Essendo condivisi, garantiscono una rigorosa simmetria di validazione tra le interfacce di creazione e di modifica.

Attributi:

Ogni attributo di questi oggetti rappresenta uno specifico campo del dominio e rispetta rigorosamente la forma definita da `FieldDefinition`:

Per `deviceFormFields`:

- + `deviceName`: `FieldDefinition`
- + `deviceOs`: `FieldDefinition`
- + `deviceDescription`: `FieldDefinition`

Per `assetFormFields`:

- + `name`: `FieldDefinition`
- + `assetType`: `FieldDefinition`
- + `description`: `FieldDefinition`

Metodi:

Trattandosi di oggetti di pura configurazione statica dei dati, non espongono alcun metodo operativo.

5.8.2) Component

5.8.2.1) AsyncButton

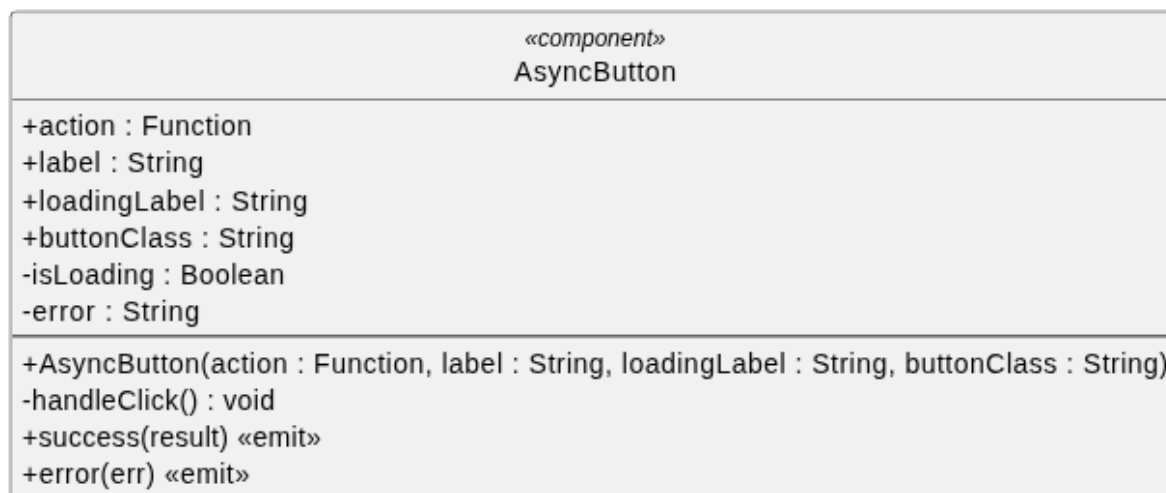


Figura 163: Componente - AsyncButton

Descrizione

Componente generico che esegue una funzione asincrona al click, gestendo lo stato di caricamento e la cattura degli errori.

Non possiede alcuna conoscenza del dominio applicativo: riceve la funzione da eseguire come parametro e comunica l'esito tramite eventi.

Attributi

- - `action: Function` : Funzione da eseguire al click del pulsante
- - `Label: String` : Testo del bottone.
- - `loadingLabel: String` : Testo del bottone durante l'esecuzione della funzione.
- - `buttonClass: String` : Stringa contenente l'elenco delle classi da applicare al pulsante
- - `isLoading : Boolean` — indica se la funzione asincrona è in esecuzione. Durante il caricamento il bottone è disabilitato per prevenire click multipli.
- `error : String` — contiene il messaggio dell'ultimo errore verificatosi, null se l'ultima esecuzione ha avuto successo.

Metodi

- + `AsyncButton(action:Function, label:string, loadingLabel:String, buttonClass:String)` — costruttore. Riceve la funzione asincrona da eseguire, il testo del bottone nei due stati (default e caricamento), e una classe CSS opzionale per lo stile.
- `handleClick()` — metodo privato invocato al click. Verifica che non sia già in corso un'esecuzione, imposta lo stato di caricamento, esegue la funzione asincrona e gestisce il risultato.
- + `success(result)` — evento emesso al completamento con successo della funzione asincrona. Trasporta il valore restituito dalla funzione.
- + `error(err)` — evento emesso in caso di fallimento. Trasporta l'eccezione catturata.

5.8.2.2) BaseModal

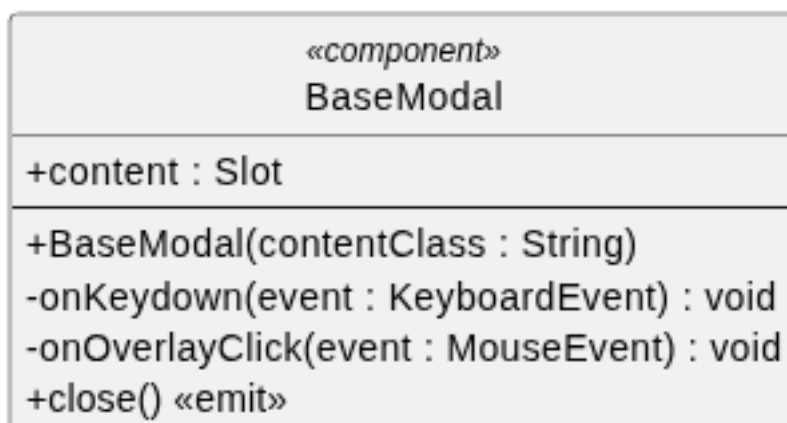


Figura 164: Componente - BaseModal

Descrizione Componente generico che fornisce l'infrastruttura per qualsiasi finestra di dialogo. Non possiede attributi interni né conosce il contenuto che ospita, interamente delegato a uno slot.

La decisione di quando il modale debba apparire o scomparire è responsabilità del componente padre, che lo monta e smonta tramite direttiva condizionale.

Attributi:

- - `ContentClass: String` : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - `Content: Slot` : slot di default che accetta il contenuto del modale. Il componente padre inietta tramite questo slot titolo, messaggio, bottoni e qualsiasi altro elemento necessario. Il `BaseModal` non ha alcuna conoscenza di ciò che lo slot contiene.

Metodi:

- + `BaseModal(contentClass:String)` : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - `onKeyDown(Event:KeyboardEvent)` : metodo privato registrato come listener globale al mount. Intercetta la pressione del tasto Escape e emette l'evento di chiusura.
- - `onOverlayClick(event)` — metodo privato invocato al click sull'overlay. Emette l'evento di chiusura solo se il click avviene sull'overlay stesso e non su un elemento figlio.
- `close()` — evento emesso quando l'utente richiede la chiusura del modale (tramite Escape o click sull'overlay). Il padre decide come reagire.

5.8.2.3) FormField

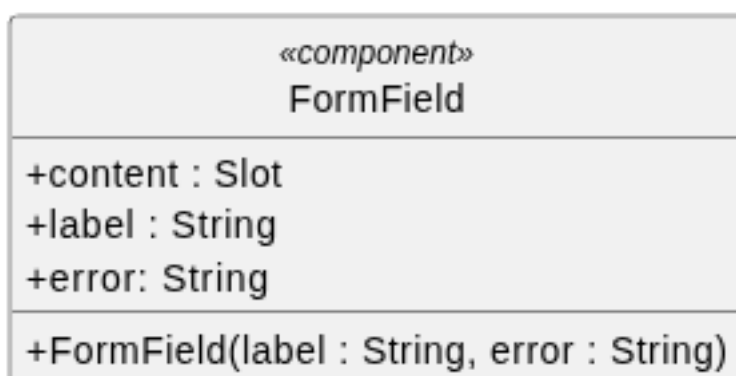


Figura 165: Componente - FormField

Descrizione:

Componente generico che funge da wrapper per un singolo campo di un form. Si occupa di mostrare la label associata al campo e l'eventuale messaggio di errore di

validazione. Non conosce il tipo di input che ospita — il campo vero e proprio (text, select, textarea, radio) è delegato allo slot.

Attributi:

- - label: String: Testo da mostrare nella label del campo di input.
- - error: String: Testo da mostrare in caso di errore.
- - Content: Slot: slot di default che accetta l'input vero e proprio. Il widget padre inietta tramite questo slot il campo appropriato con il relativo binding ai dati.

Metodi:

- + FormField(label:String, error: String):

Costruttore. Riceve il testo della label e l'eventuale messaggio di errore. Se error è null il campo è considerato valido e il messaggio non viene mostrato.

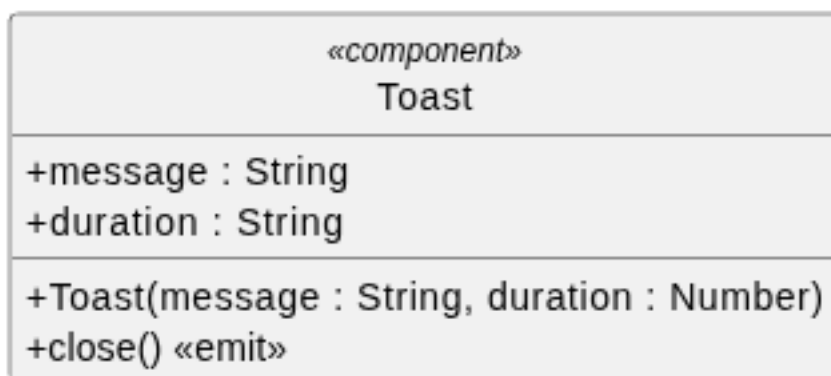
5.8.2.4) Toast

Figura 166: Componente - Toast

Descrizione:

Componente generico che mostra un messaggio temporaneo a schermo sotto forma di notifica. Scompare automaticamente dopo un tempo definito senza richiedere interazione da parte dell'utente. Non mantiene stato interno — il padre lo monta quando serve mostrare un messaggio e lo smonta quando riceve l'evento di chiusura.

Attributi:

- - message:String: messaggio da mostrare.
- - duration:Number: durata a schermo del componente.

Metodi:

- + Toast(message:String, duration:Number): costruttore. Riceve il testo del messaggio e la durata in millisecondi prima della chiusura automatica. Al mount del componente viene avviato un timer che, allo scadere, emette l'evento di chiusura.